Lecture 5: The Stack

UNIVERSITY
OF MIAMI



EEN 312: Processors: Hardware, Software, and Interfacing

Department of Electrical and Computer Engineering
Spring 2014, Dr. Rozier (UM)

UNIVERSITY
OF MIAMI



COURSE PLAN FOR TODAY

1. Midterm I
2. The Stack and Memory

UNIVERSITY
OF MIAMI



MIDTERM I

Midterm I

- Thursday, February 13th!
  - Two weeks away
- Covers the following Material:
  - Chapters 1 – 3
  - Laboratory 1

THE STACK AND MEMORY

UNIVERSITY
OF MIAMI



A Call Chain

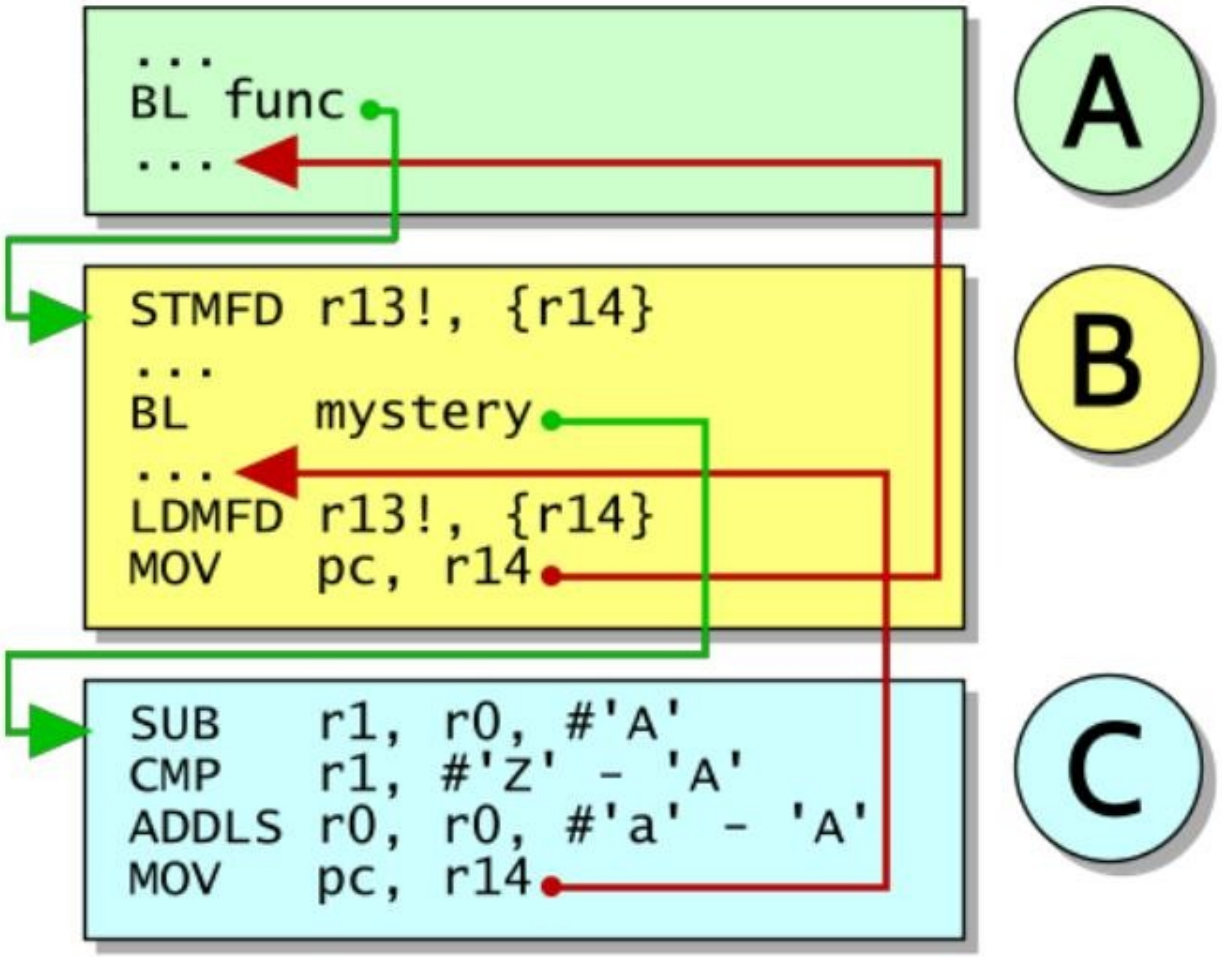


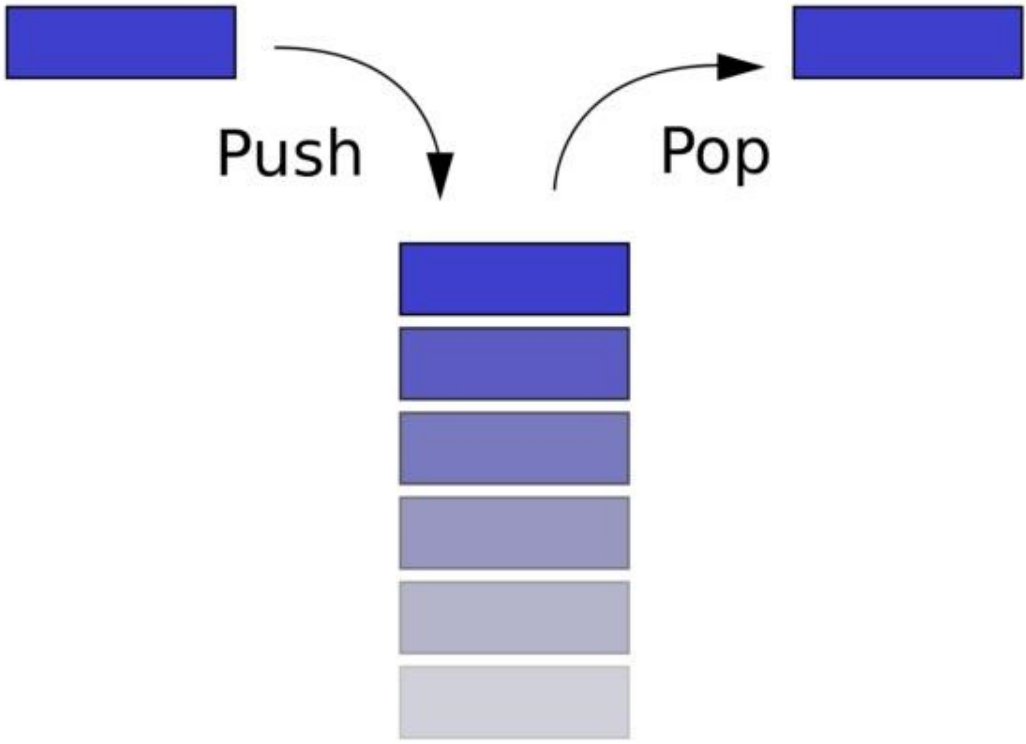
Image by David Thomas

Procedure Calling

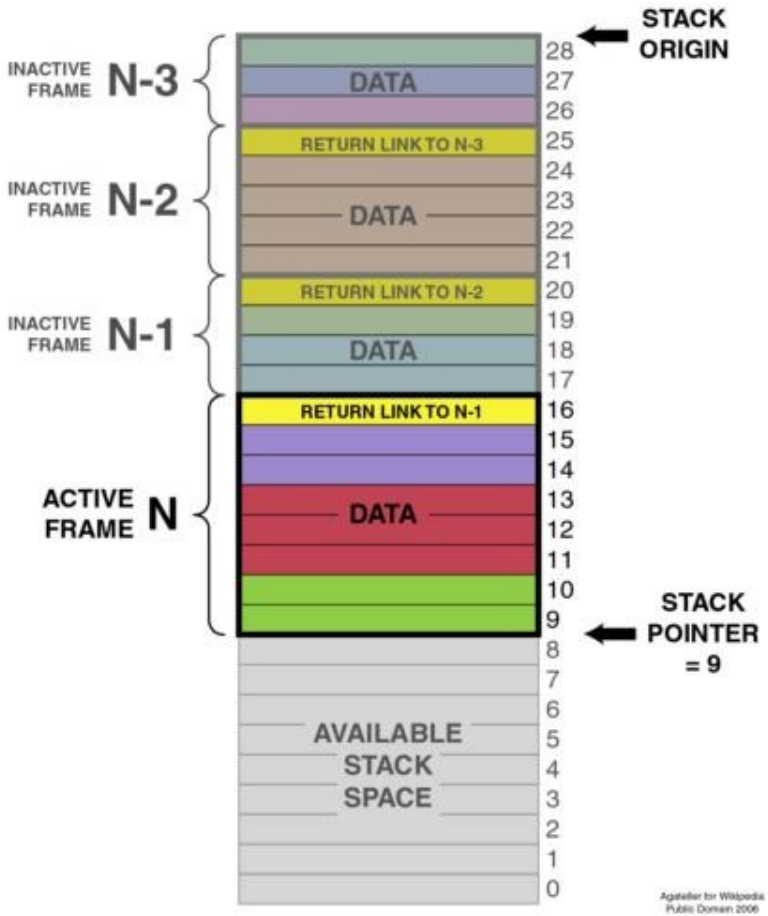
1. Place parameters for procedure in registers
2. Transfer control to procedure
3. Procedure acquires storage
4. Procedure performs function.
5. Procedure places return value in appropriate register
6. Return control

The Stack

- Region of memory managed with stack discipline.
- Accessed with push and pop

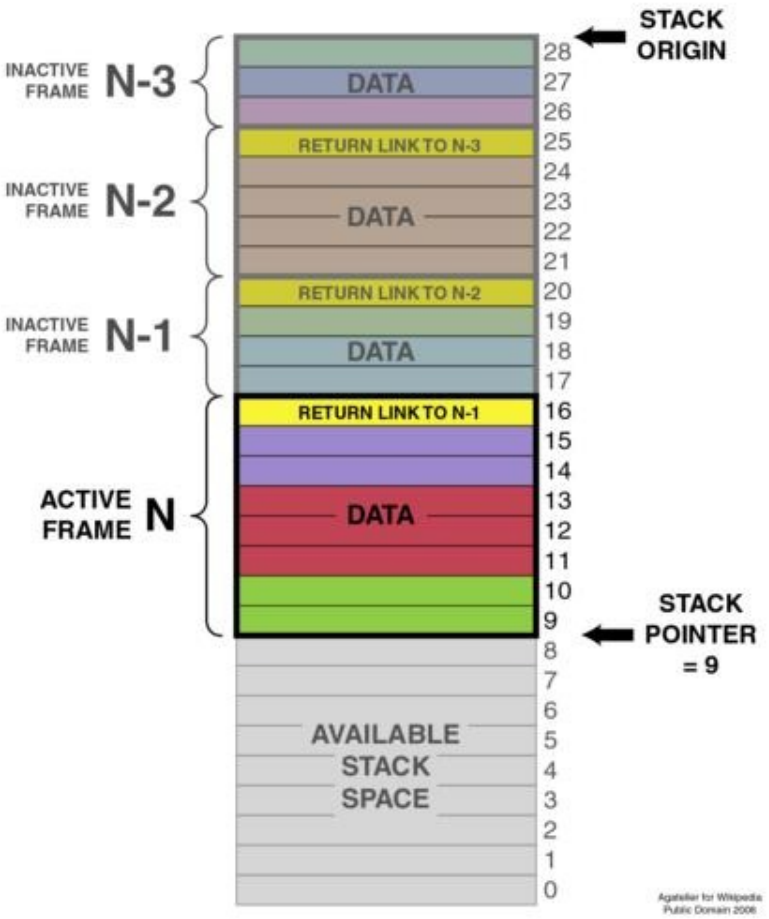


The Stack


INACTIVE FRAME N-3
DATA
RETURN LINK TO N-3
INACTIVE FRAME N-2
DATA
RETURN LINK TO N-2
INACTIVE FRAME N-1
DATA
RETURN LINK TO N-1
ACTIVE FRAME N
DATA
AVAILABLE STACK SPACE
STACK ORIGIN
STACK POINTER = 9
28
27
26
25
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
Agateller for Wikipedia
Public Domain 2006

Stack Frames

- Stack frames “belong” to a procedure.
- Store local variables here (they go out of scope automatically)
- Can communicate with other procedures with a stack frame.



Procedures and Register Use

- What if we want to use some registers in the procedure?

- Caller could have data in it!

Stack Discipline and ABI

- Stack Discipline is important
- Define an Application Binary Interface
  - How should procedures communicate? How should the be called?
- Consistency, following standards, is the key.

Conventions and Discipline

Caller

- Caller saves temporary values in its frame before the call.
- Caller restores values in its frame after the call.

Callee

- Callee saves temporary values in its frame before using.
- Callee restores values in its frame after using.

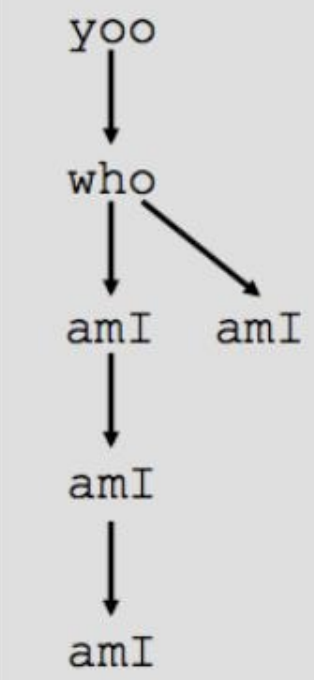
```
yoo(...)
{
   •
   •
   who();
   •
   •
}
```

```
who(...)
{
   • • •
   amI();
   • • •
   amI();
   • • •
}
```

```
amI(...)
{
   •
   •
   amI();
   •
   •
}
```

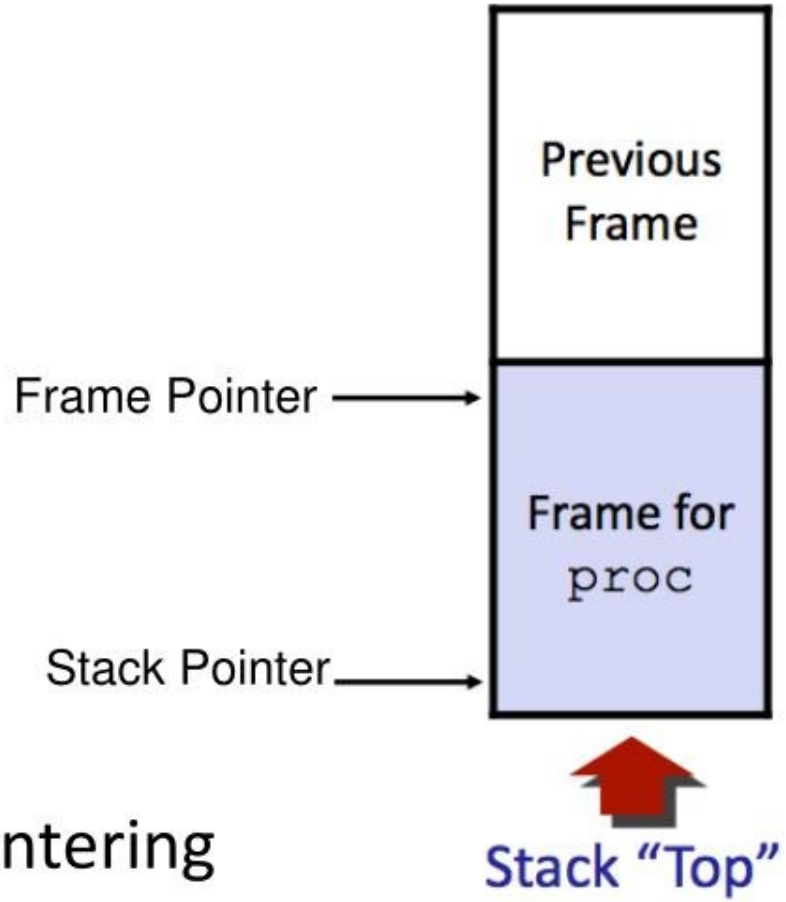
Procedure `amI()` is recursive

Example
Call Chain

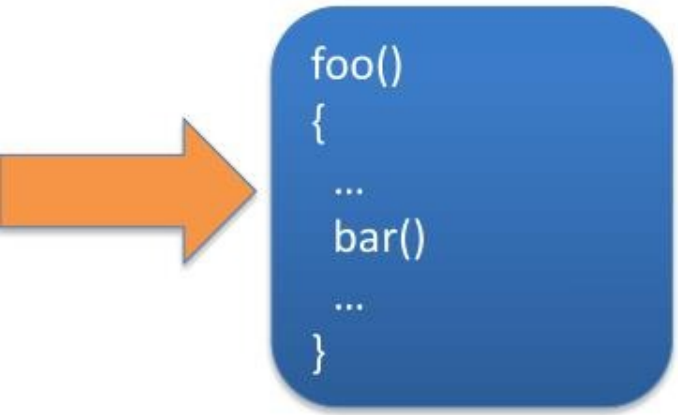


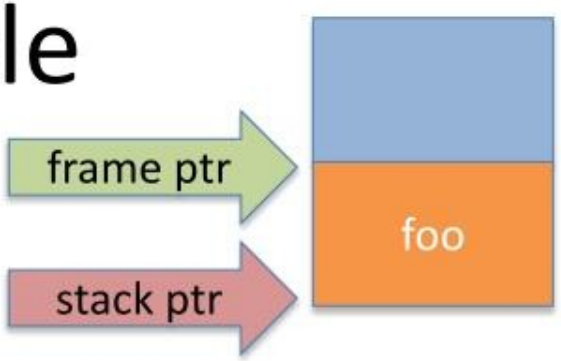
Stack Frames

- Contents
  - Local variables
  - Return information
  - Temporary space
- Management
  - Space is allocated when entering
    - Set-up code
  - Deallocated when returning
    - Finish code



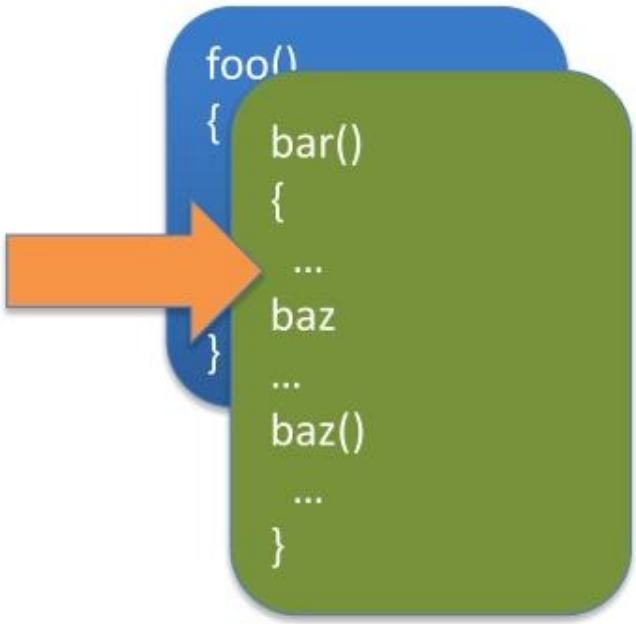
Example

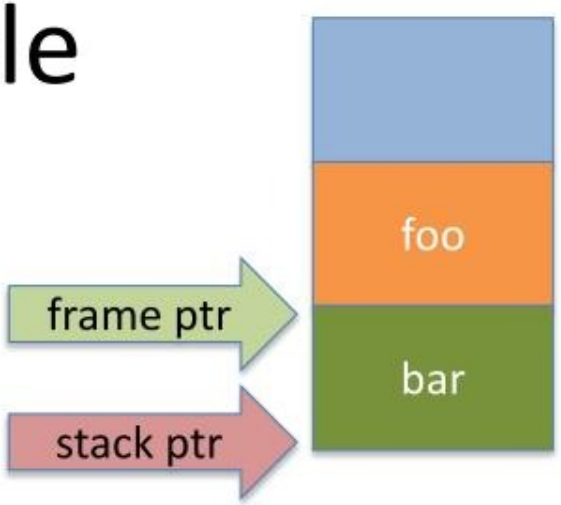




foo()

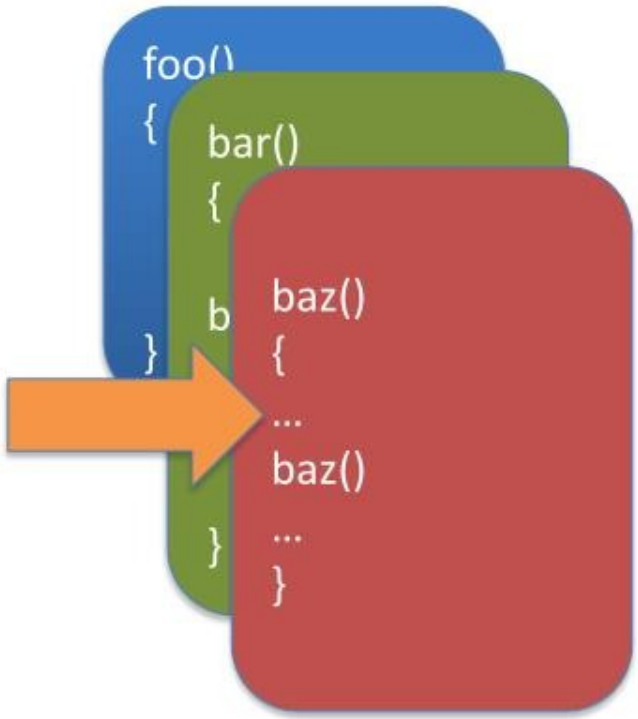
Example

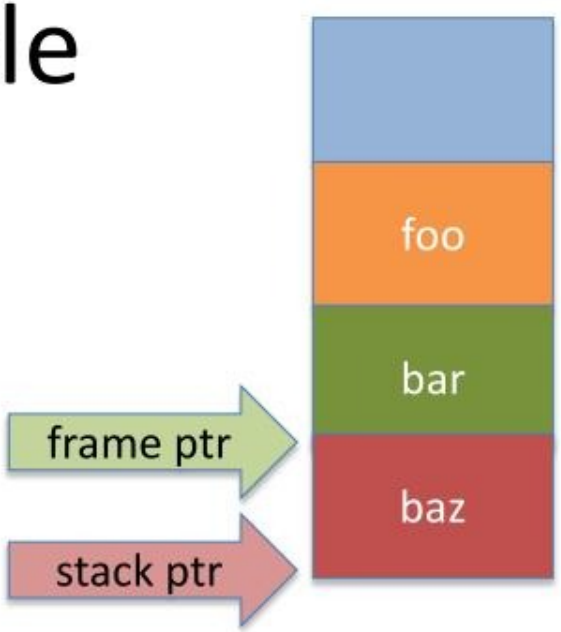




foo() ⟶ bar()

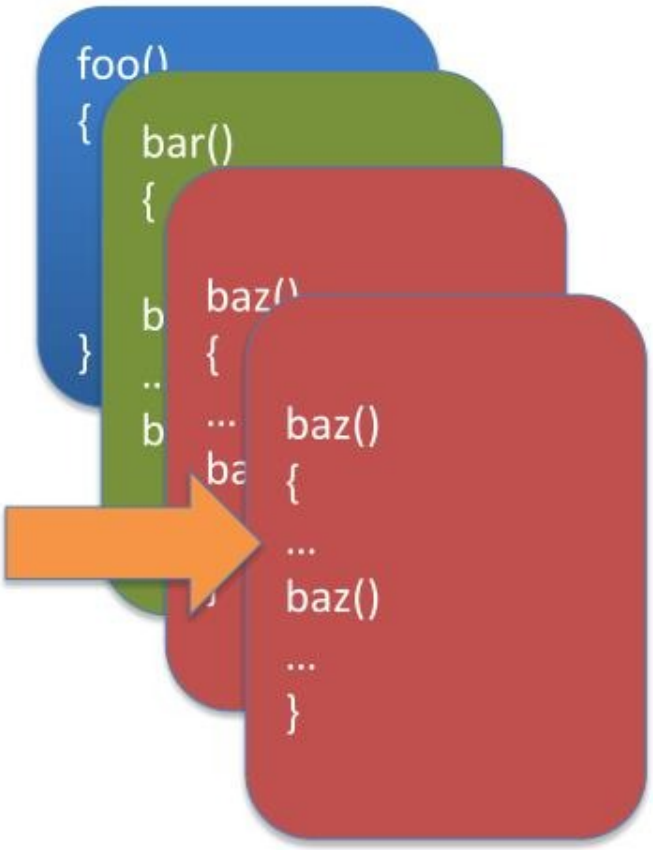
Example

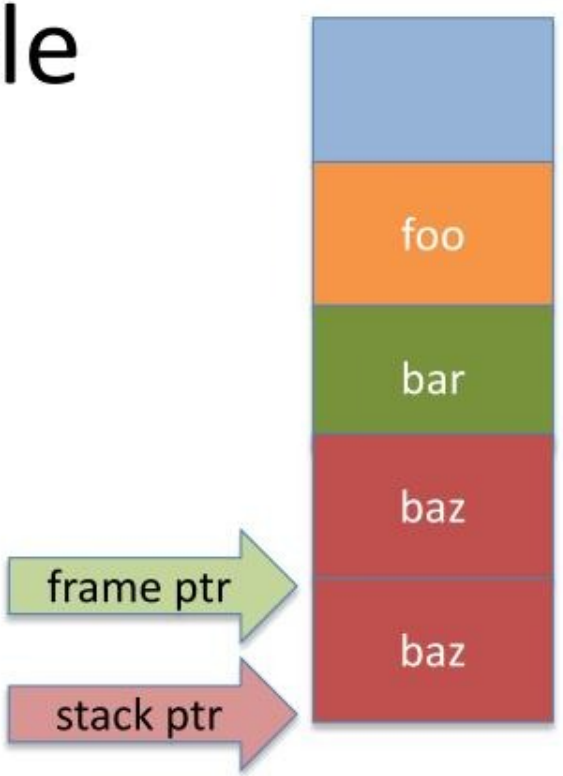




foo() → bar() → baz()

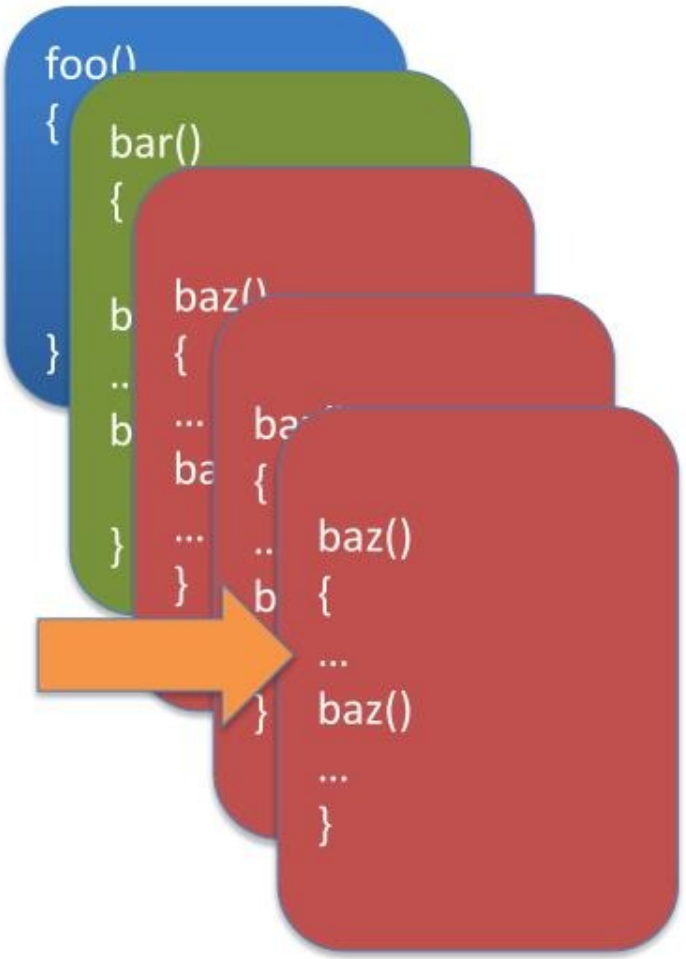
Example

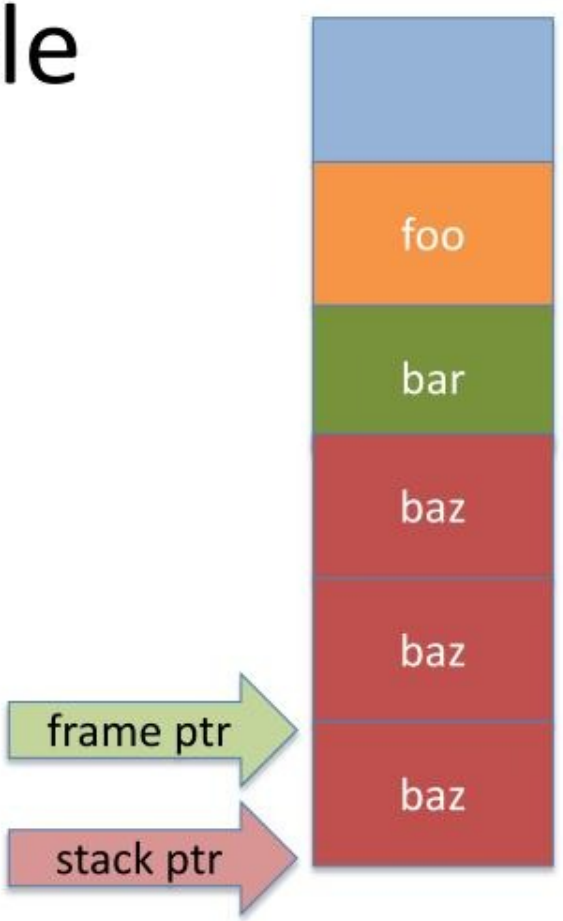




foo() → bar() → baz() → baz()

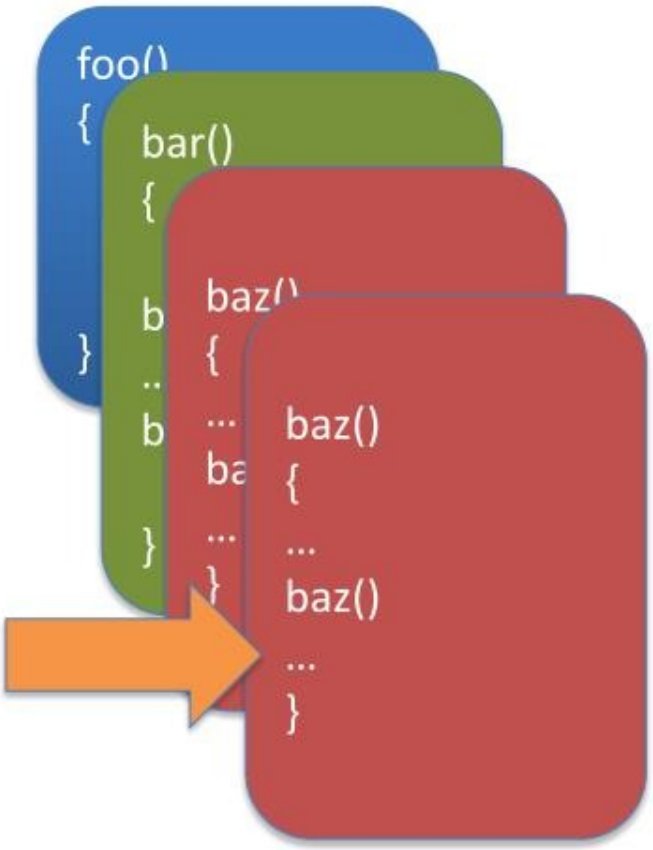
Example

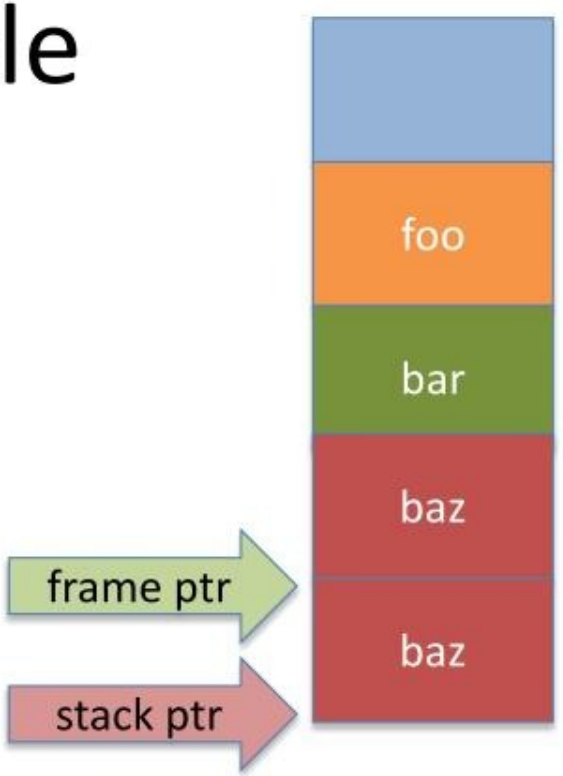




foo() → bar() → baz() → baz() → baz()

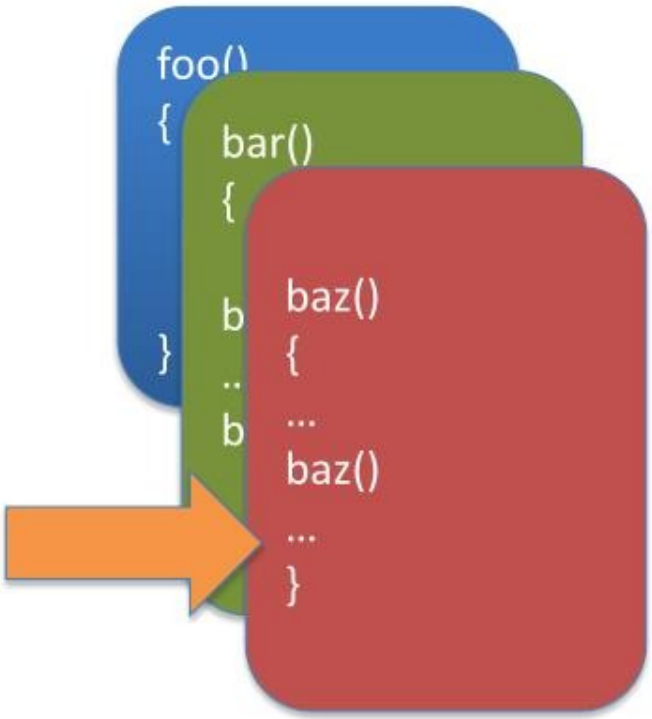
Example

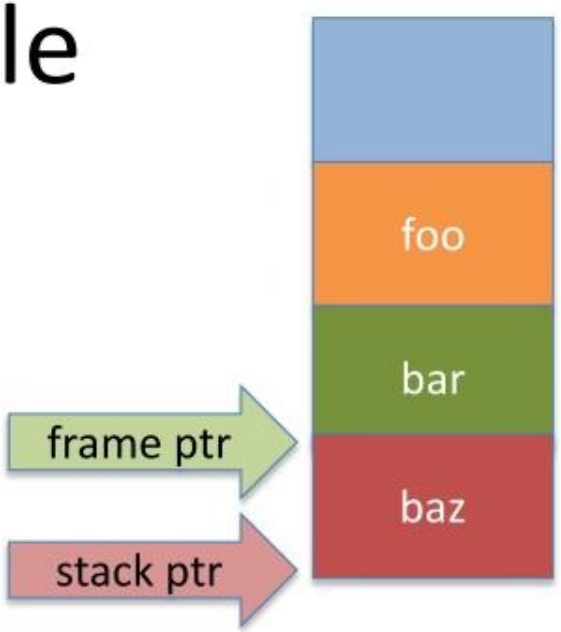




foo() → bar() → baz() → baz() → baz()

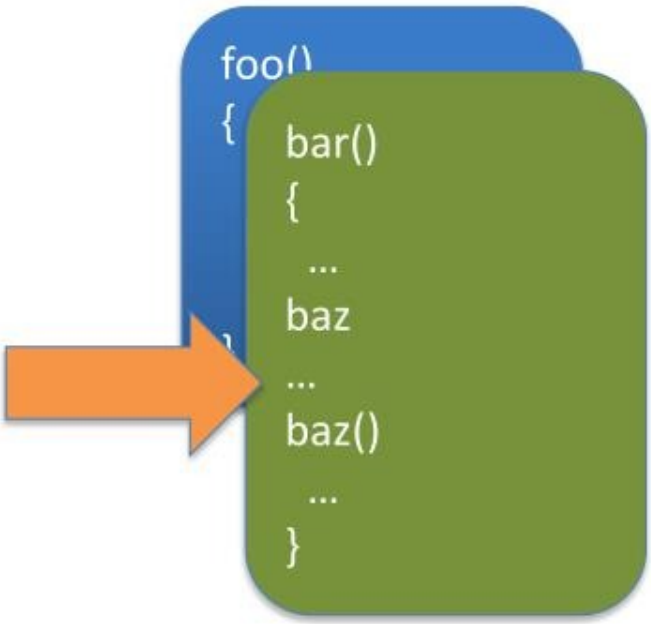
Example

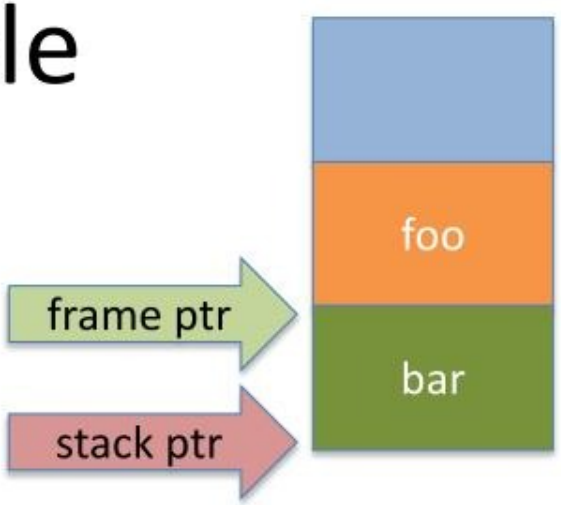




foo() → bar() → baz() → baz() → baz()

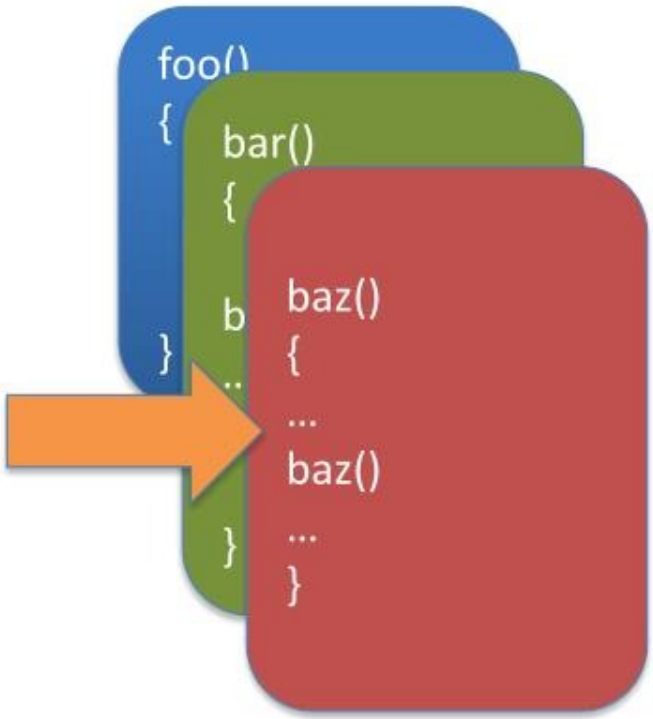
Example

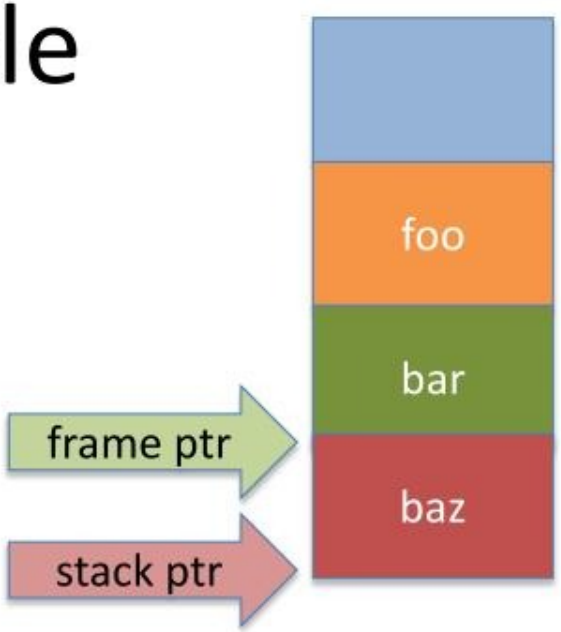


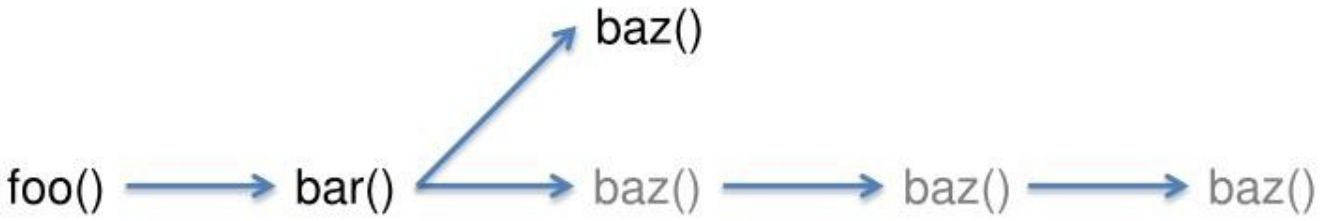


foo() → bar() → baz() → baz() → baz()

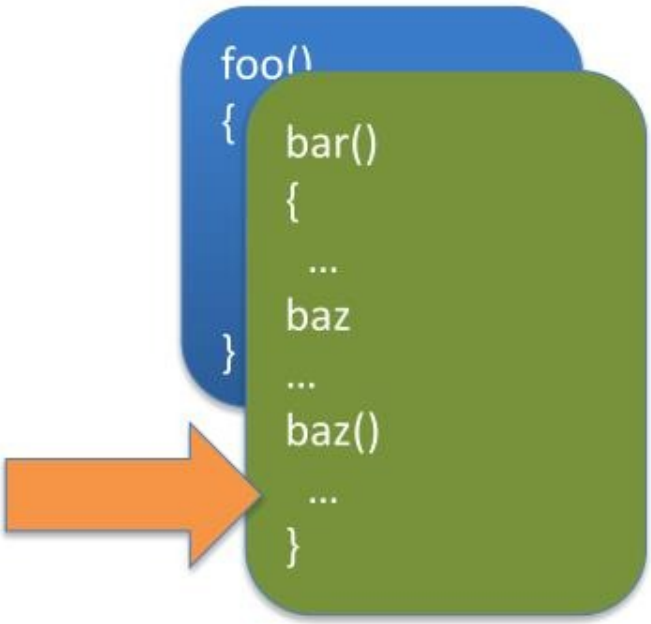
Example

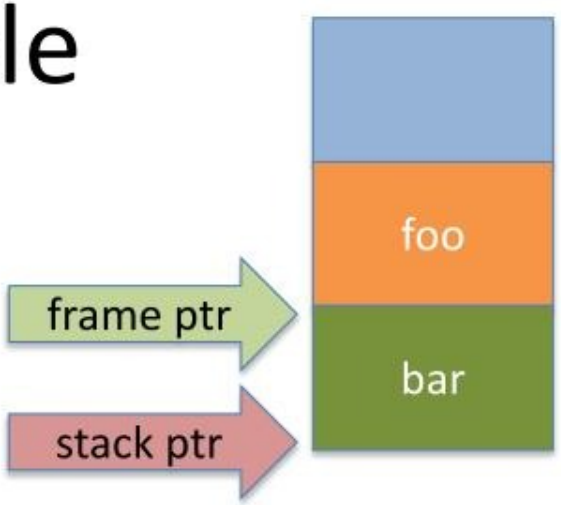

foo()
{
bar()
{
b
}
baz()
{
...
baz()
...
}

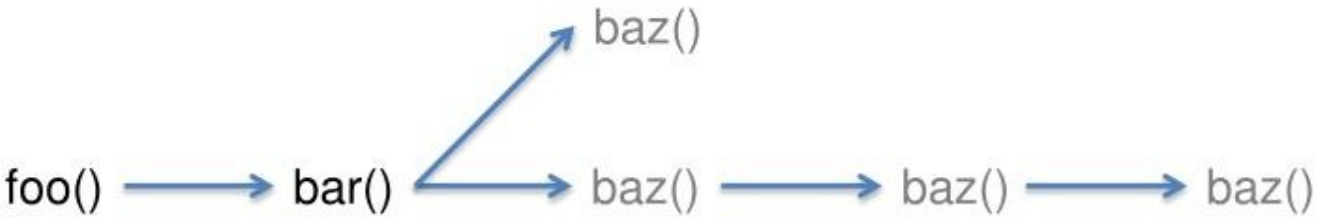

foo
bar
baz
frame ptr
stack ptr


baz()
foo()
bar()
baz()
baz()
baz()

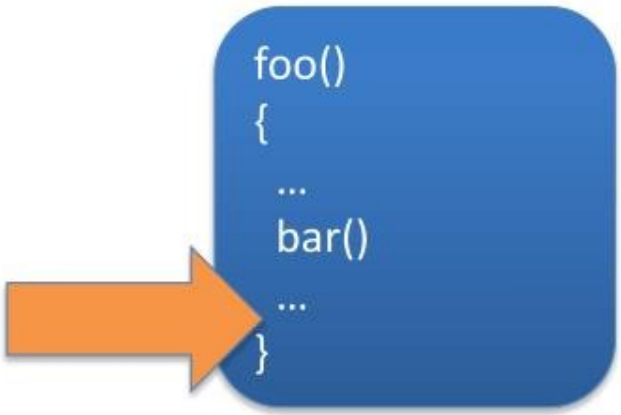
Example

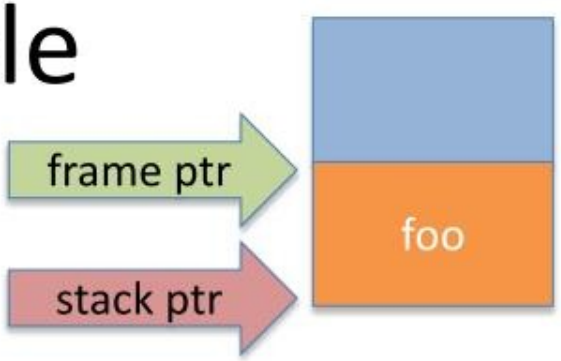

foo()
{
}
bar()
{
...
baz
...
baz()
...
}

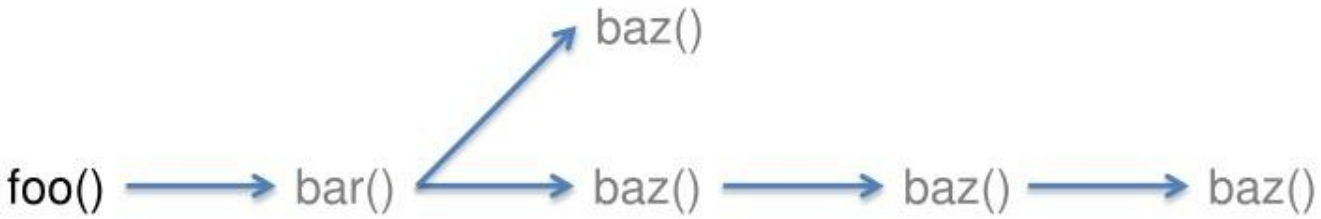

frame ptr
stack ptr
foo
bar


foo()
bar()
baz()
baz()
baz()
baz()

Example



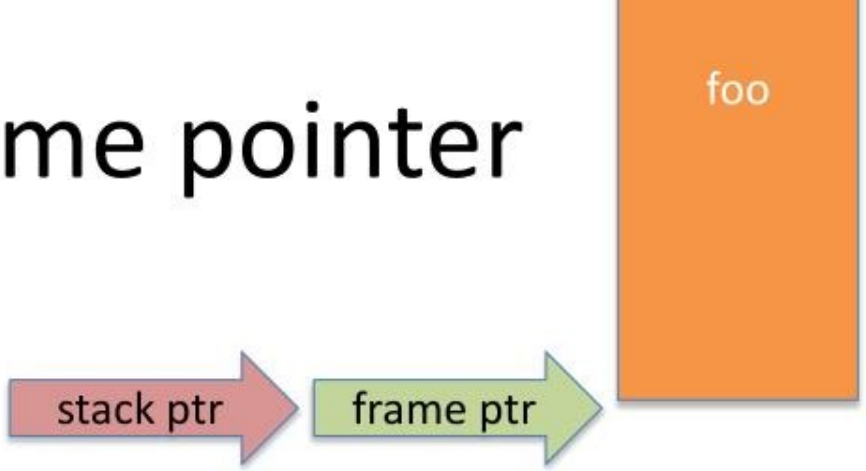




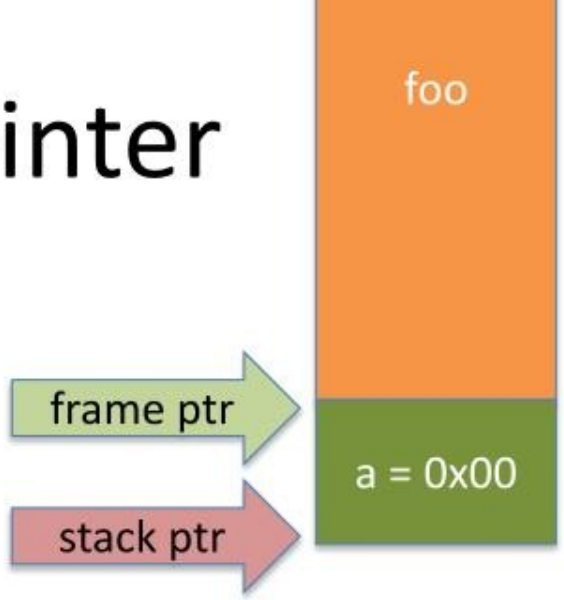
What is the frame pointer used for?

- Given a caller and a callee, the callee will begin with:
  - fp = sp
- A callee will use the stack to:
  - Save the current sp, lr, and fp before calling a function
  - Store the arguments for, and return value space before calling a function
  - Save used registers before calling a function
  - Store variables local to the procedure

Frame pointer

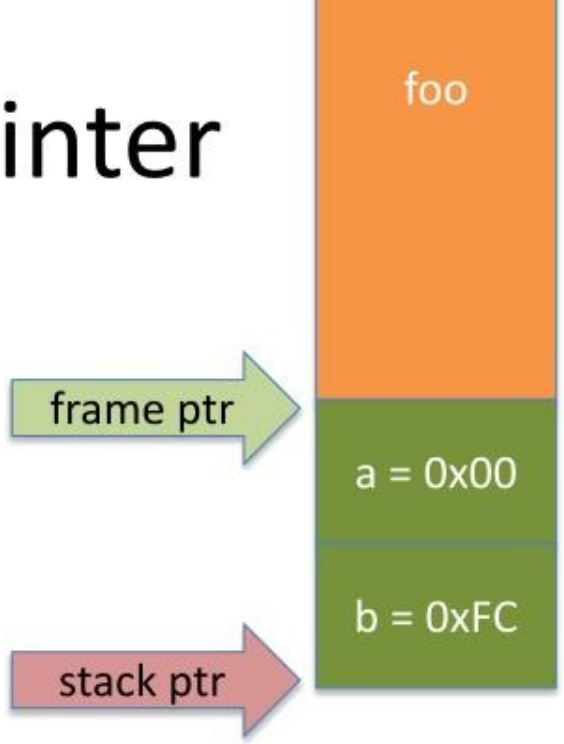


Frame pointer



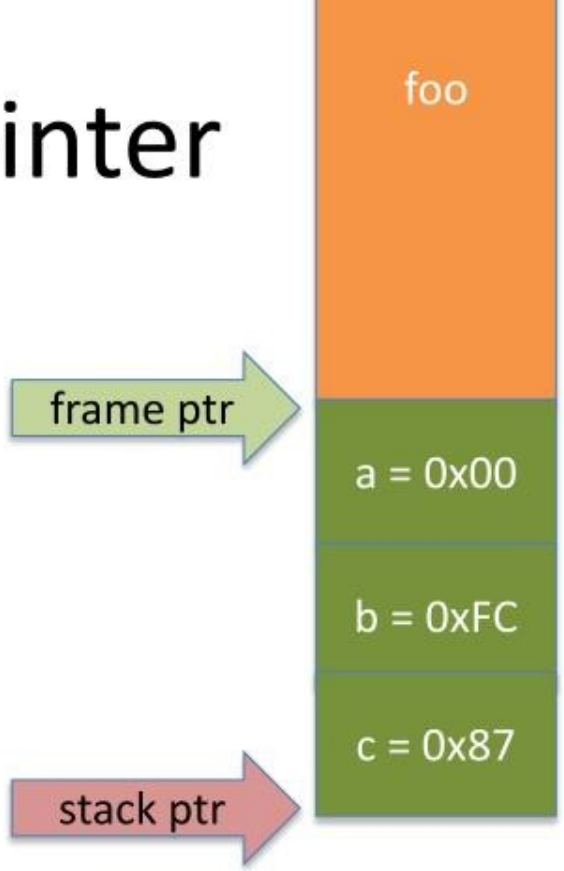
PUSH

Frame pointer



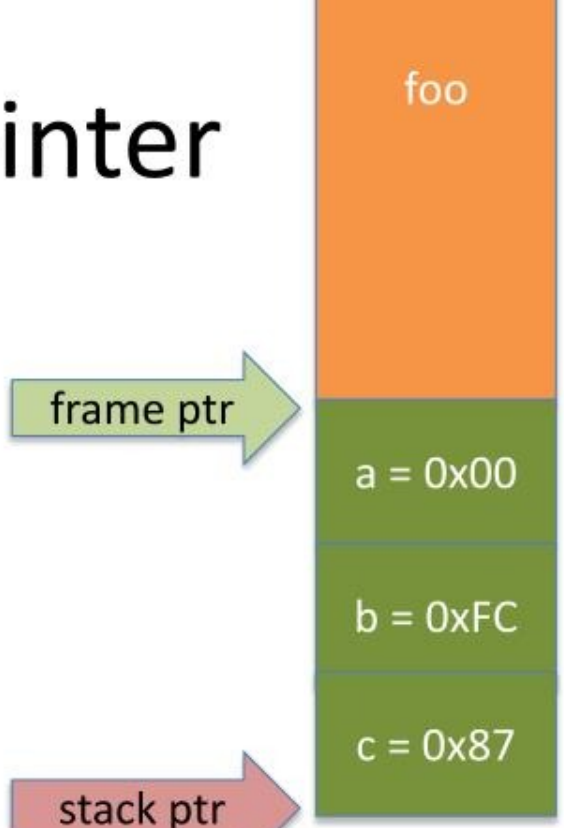
PUSH

Frame pointer



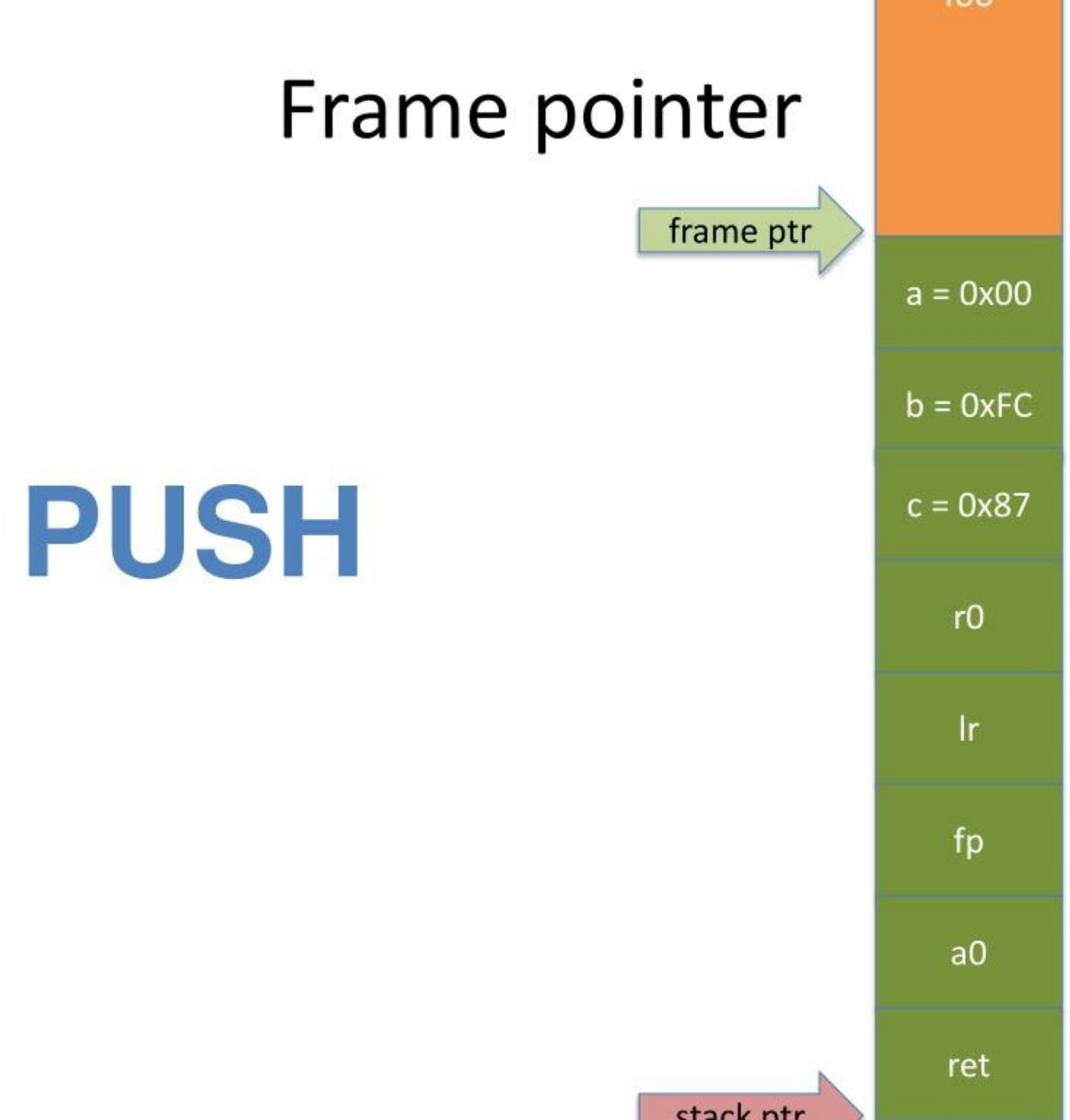
PUSH

Frame pointer



PREPARE TO CALL

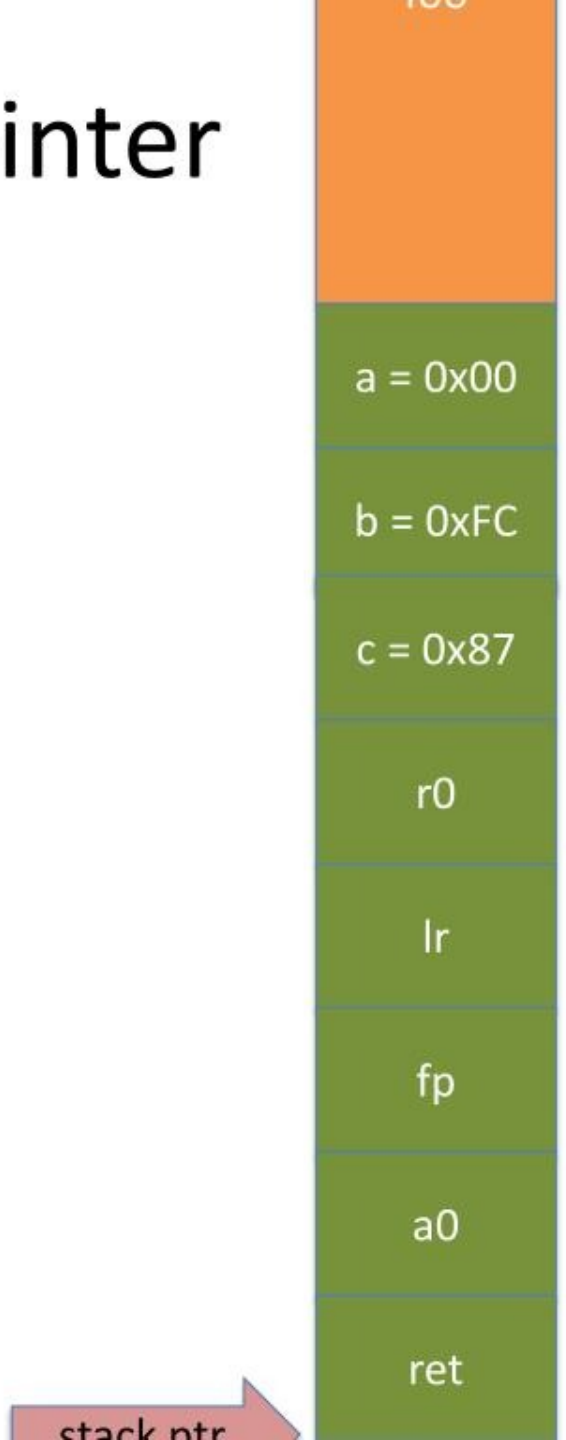
Frame pointer



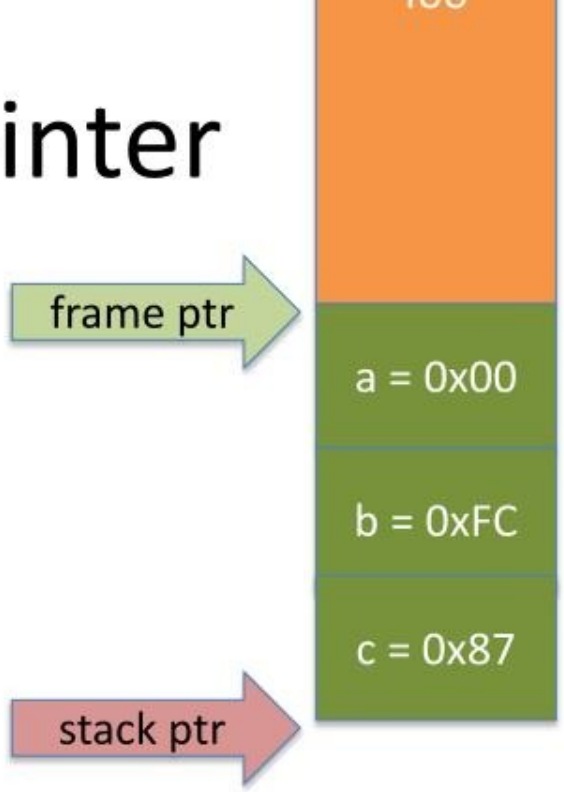
PUSH

Frame pointer

RETURN FROM CALL



Frame pointer

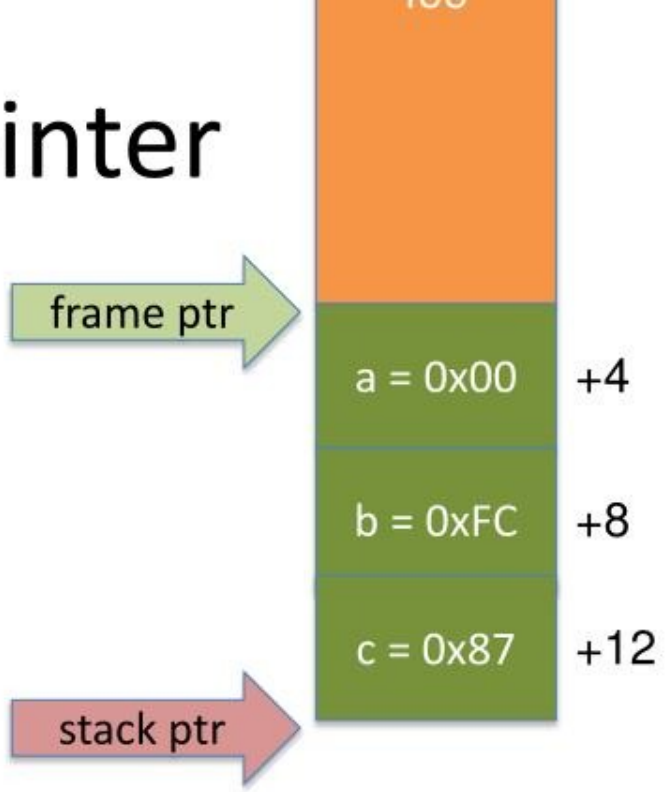


POP!

5 minute paper

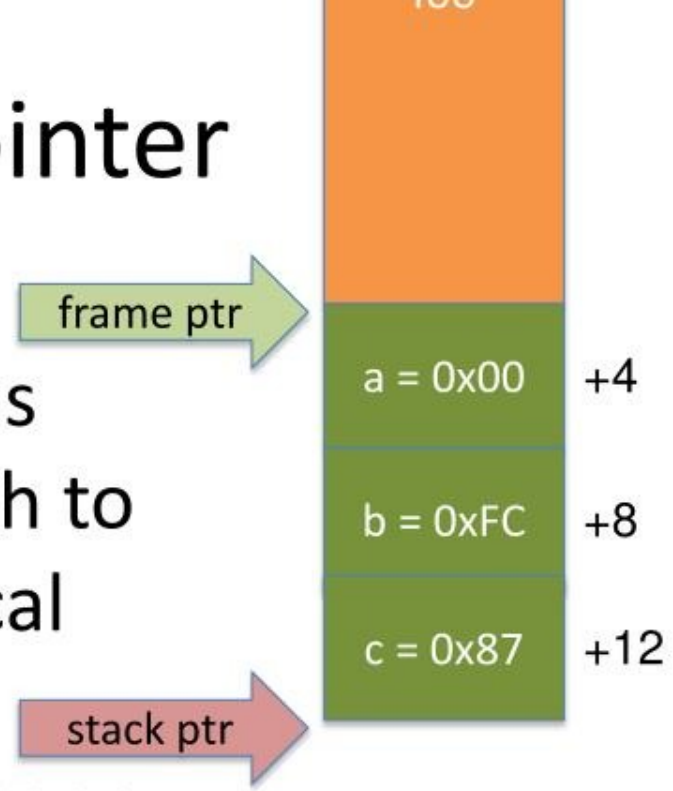
- Take 5 minutes to think about these tasks. Determine why a frame pointer is helpful

Frame pointer

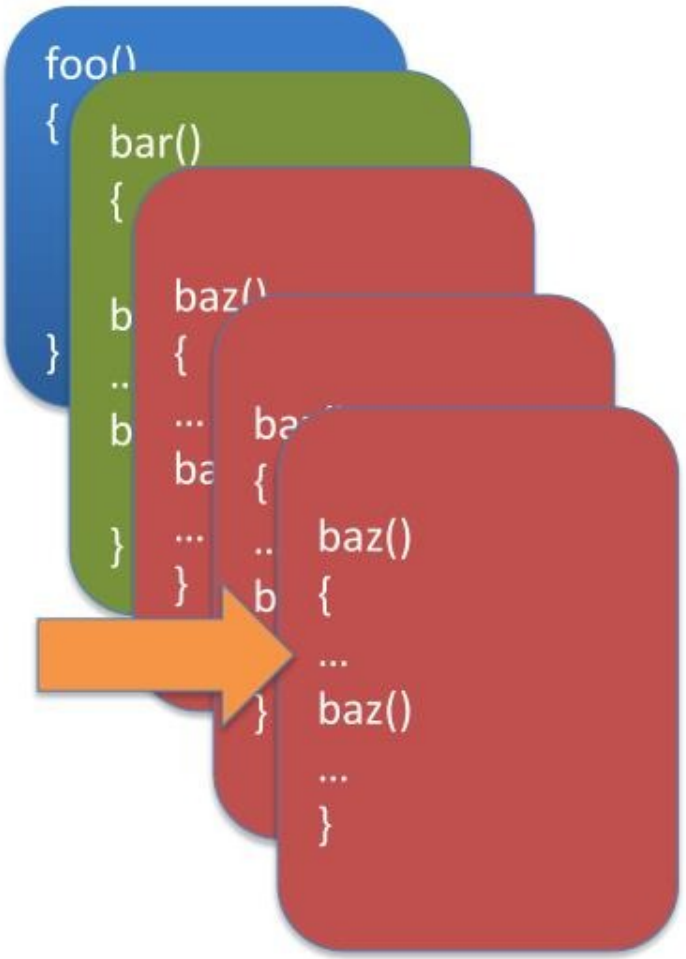


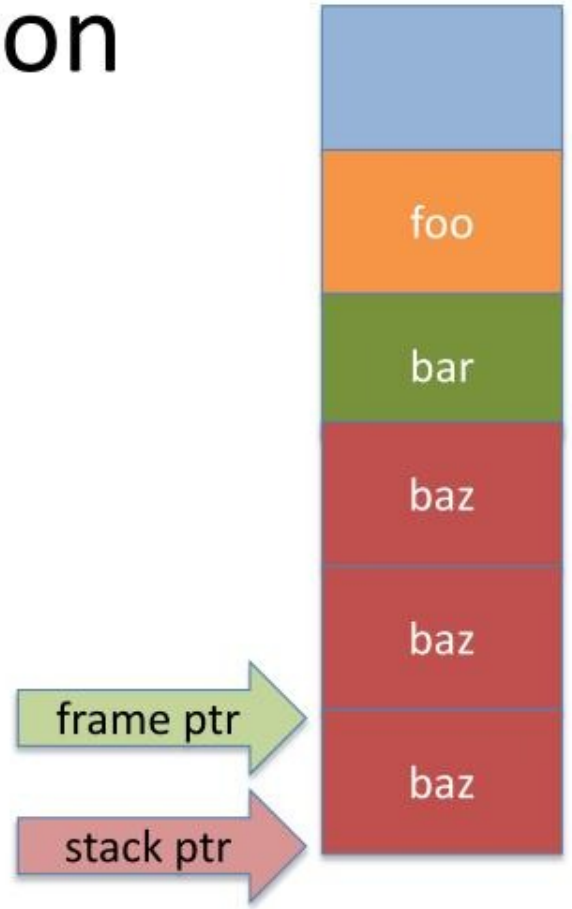
Frame pointer

- The frame pointer gives us a base address from which to offset in our frame for local storage that will automatically go out of scope.



Recursion

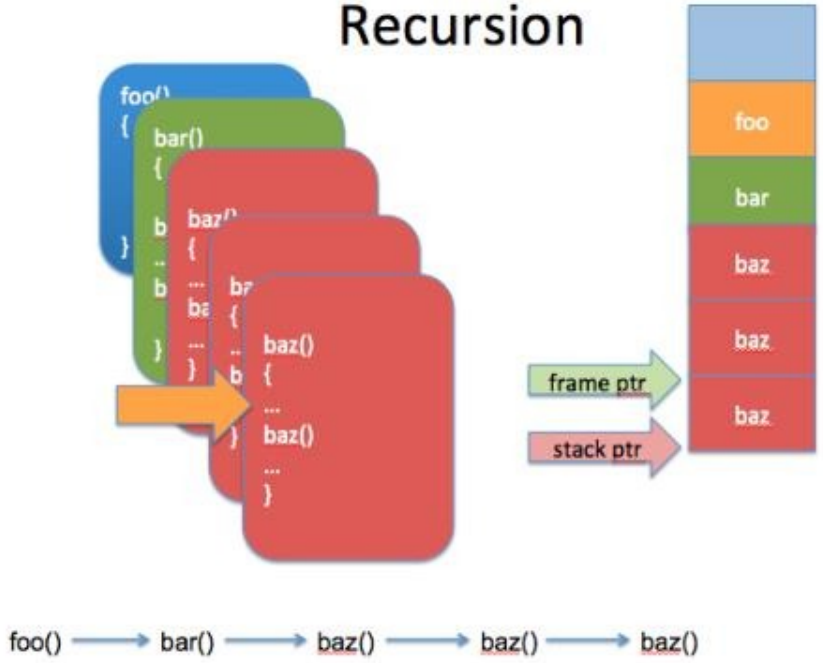




foo() → bar() → baz() → baz() → baz()

Recursion

- Stack frames give each function private storage
  - Saved registers
  - Local variables
  - Return portions
- Recursion is handled without special consideration.
- Following stack discipline and call/return pattern is critical



PUSH and POP

- PUSH<cond> {reglist}
- POP<cond> {reglist}

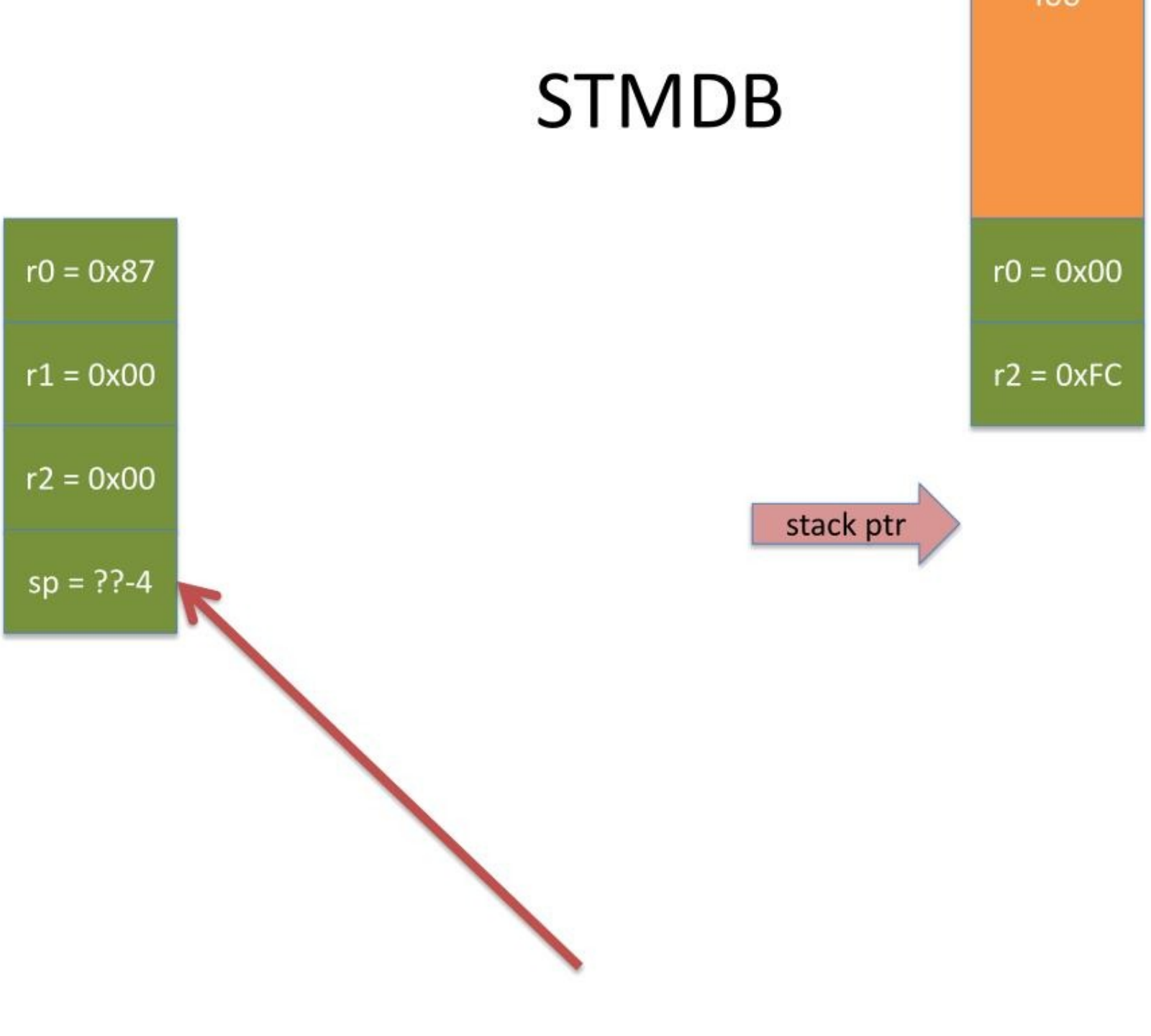
- Really pseudo-instructions!

- PUSH – STMDB
  - Store multiple registers, decrement address before access.
- POP – LDMIA
  - Store multiple registers, increment address after access.

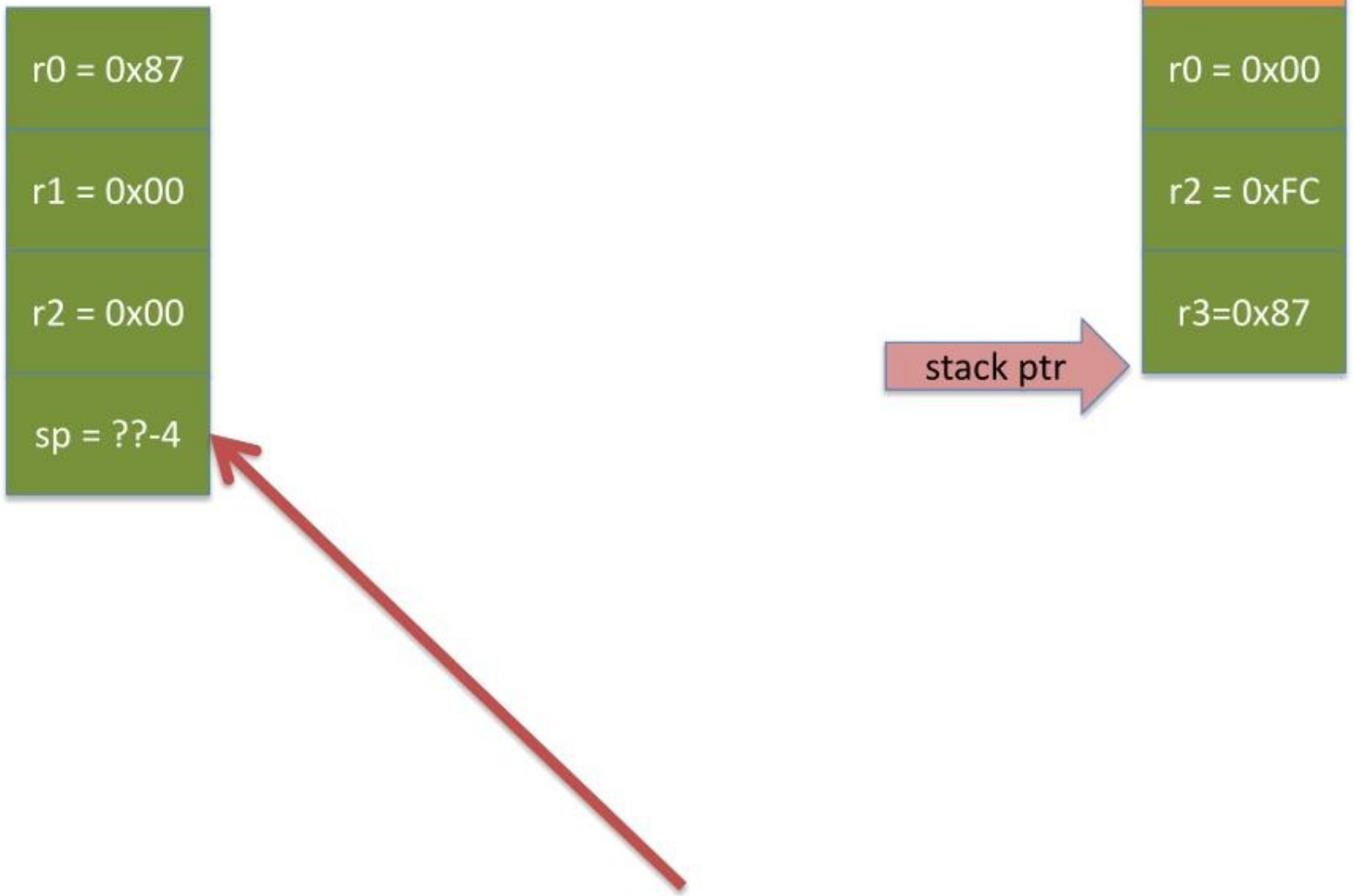
STMDB



STMDB



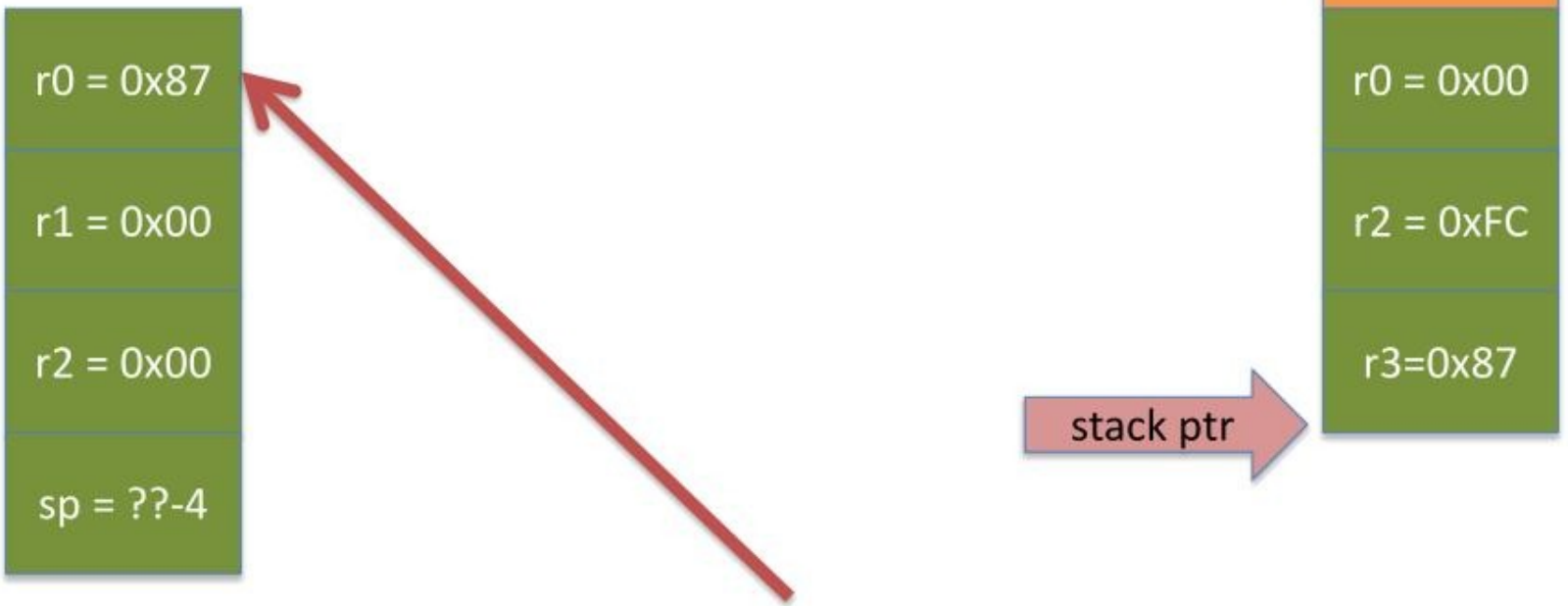
STMIA



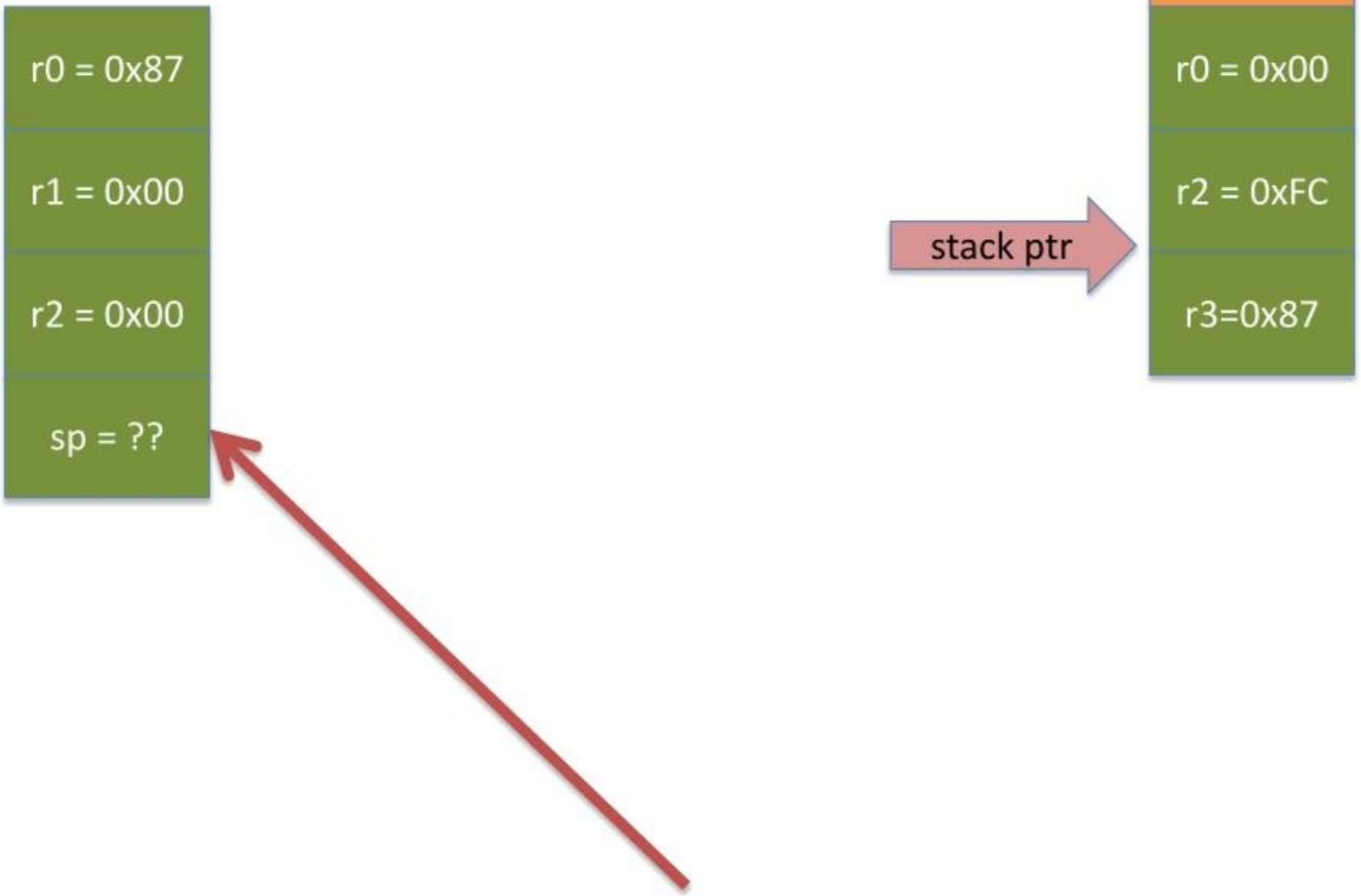
STMDB



LDMIA



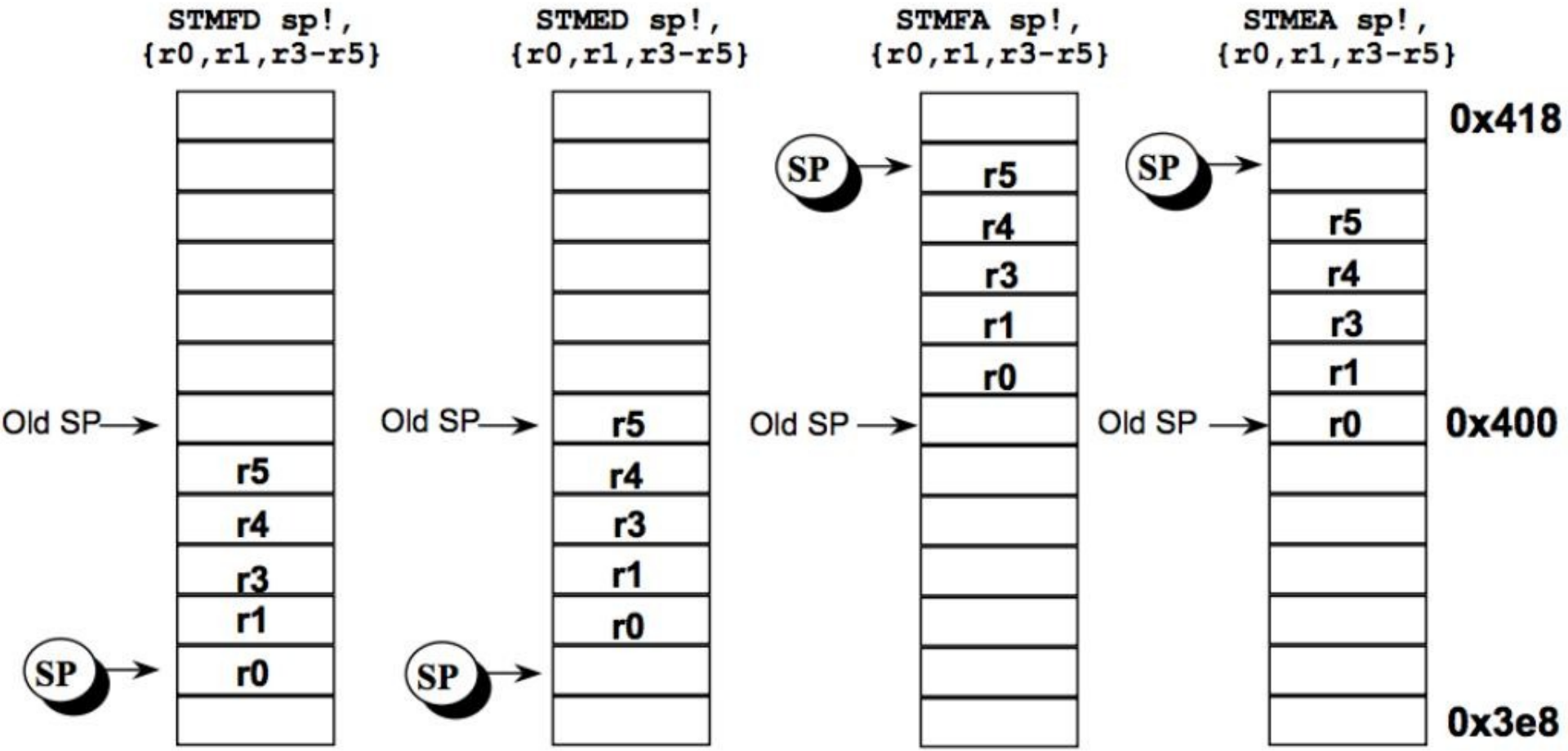
LDMIA



Multiple Ways to Handle the Stack

- Stack Pointer can
  - Point to the lass occupied stack (Full stack)
    - (needs pre-decumenting)
  - Point to the next address to be occupied (Empty stack)
    - (needs post-decrementing)
- Stack type can be given as a postfix to the instruction:
  - STMFD/LDMFD
  - STMFA/LDMFA
  - STMED/LDMED
  - STMEA/LDMEA

Stack Examples

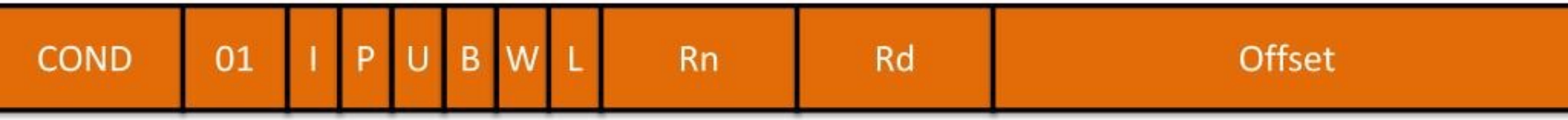


RISC Mentality

- There are no special functions for stacks!

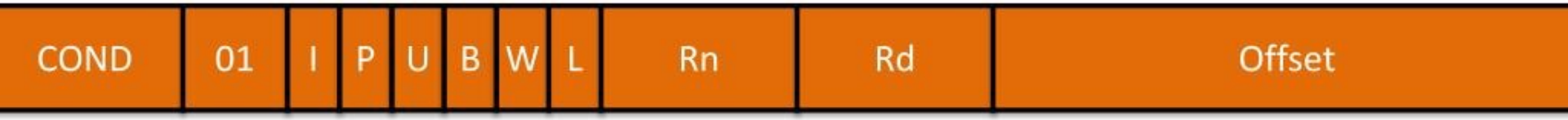
- Let's look at stack instructions:
  - STMDA/LDMDA

Single Data Transfer



- Interpreting
  - Cond – 4 bits, condition field
  - I – immediate offset
  - Pre/post indexing bit (add offset before(0)/after(1) transfer)
  - Up/down bit (subtract(0)/add(1) offset to base
  - Byte/word bit (transfer word(0)/byte(1))
  - Write-back bit (1 to write address back into base)
  - Load store bit (0 store to memory, 1 load from memory)
  - Rn – base register
  - Rd – source/destination register

Single Data Transfer



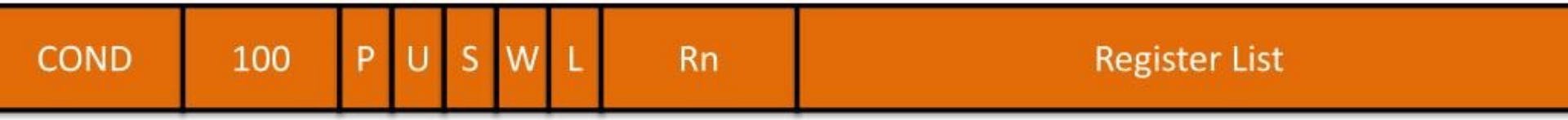
- Interpreting
  - Cond – 4 bits, condition field
  - I – immediate offset
    - 1 – offset is immediate

Single Data Transfer



- Interpreting
  - Cond – 4 bits, condition field
  - I – immediate offset
    - 0 – offset is register

Block Data Transfer



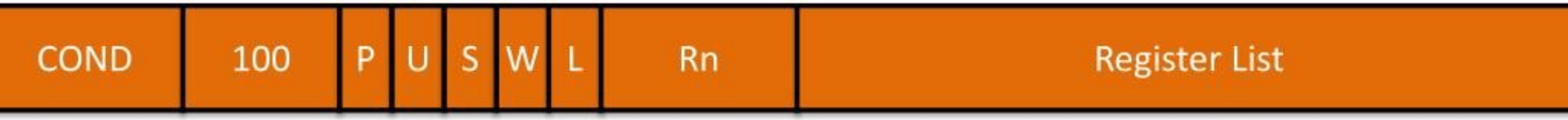
- Interpreting
  - Cond – 4 bits, condition field
  - I – immediate offset
  - Pre/post indexing bit (add offset before(0)/after(1) transfer)
  - Up/down bit (subtract(0)/add(1) offset to base
  - PSR & force user bit (0 do not load PSR or force user mode, 1 load PSR and force user mode)
  - Write-back bit (1 to write address back into base)
  - Load store bit (0 store to memory, 1 load from memory)
  - Rn – base register
  - Register List

LDR/LDR LDM/STM

- Load or Store single, or multiple

- Write back, W-bit
  - ldr Rd, [Rn, Offset]
  - ldr Rd!, [Rn, Offset]

Block Data Transfer



- Register list, how to implement?
  - 16 bit field

UNIVERSITY
OF MIAMI



WRAP UP

For next time

- Read Chapter 3, Sections 3.1 – 3.5, 3.8, and 3.9
- Arithmetic for Computers
- Midterm 1 Approaching!
  - February 13th

UNIVERSITY OF MIAMI

